

GamerStore ERP

Guía completa para la exposición — 6 partes + flujos explicados

Panel de administración de una tienda de tecnología · Spring Boot (API REST) + React (Vite) + MySQL · Seguridad con JWT
Incluye: cómo funciona cada módulo paso a paso, ejemplos reales y “qué usamos para qué”.

El viaje de CUALQUIER petición (memorícenlo — todo el ERP funciona así)

IDA: Pantalla admin (React) → endpoints.js → client.js (+token) → Controller (/api) → Service (reglas) → Repository → Base de datos
VUELTA: Base de datos → Repository → Service → Mapper (Entidad+DTO) → Controller (JSON) → client.js → setState → la pantalla se actualiza

En palabras: el usuario toca un botón → el front pide a una URL `/api/...` con su token → el **Controller** recibe → el **Service** aplica las reglas → el **Repository** lee/graba en la **base de datos** → se arma un **DTO** (JSON) → el front actualiza la pantalla.

#	Parte (a exponer)	Responsable	De qué trata
1	Arquitectura y modelo de datos	_____	Las capas del sistema y las entidades que se vuelven tablas
2	Datos reales + imágenes	_____	La carga inicial de datos y las imágenes guardadas en el proyecto
3	Seguridad: login JWT + sesión	_____	Login con token, protección del panel, refresco y contador
4	Productos y Categorías	_____	ABM del catálogo con subida de imágenes
5	Clientes (RENIEC), Pedidos, PDF	_____	DNI real, registrar ventas y descargar el reporte
6	Dashboard, Usuarios, validaciones	_____	Estadísticas, gestión de usuarios y anti-duplicados

 **Truco para las preguntas del profe:** todos los módulos siguen el MISMO viaje de arriba. Si te preguntan por algo que no es “tu parte”, explica ese viaje y di en qué archivo vive; con eso nadie se queda callado.



¿Qué usamos y para qué? — chuleta por si pregunta el profe

"Para X usamos Y, que sirve para Z"

Seguridad

- **Autenticación:** usamos **Spring Security** + **JWT** (librería **jjwt**). Al iniciar sesión se emite un **token firmado** que el servidor verifica en cada petición, sin guardar sesión en memoria (stateless).
- **Contraseñas:** usamos **BCrypt** para **hashear** las claves; nunca se guardan en texto plano.
- **Sesión revocable:** usamos **refresh tokens guardados en la base de datos** (tabla `refresh_token`) con **rotación**, para poder cerrar sesión de verdad (revoke) y renovar el acceso cada 5 min sin re-loguear.
- **Autorización:** usamos **roles** (ADMIN) y reglas por ruta en **SecurityConfig**; un **filtro** (`JwtAuthenticationFilter`) valida el token en cada request; `/api/admin/**` exige rol ADMIN.
- **CORS:** configurado para permitir que el front (:5173) llame a la API (:8080).

Backend

- **Lenguaje:** Java 17. **Framework:** Spring Boot 3.5.6.
- **API REST:** Spring Web (`@RestController`) para exponer los endpoints `/api/...`.
- **Acceso a datos (ORM):** Spring Data JPA con **Hibernate** — las clases `@Entity` se vuelven tablas y los **repositorios** generan las consultas por el nombre del método (sin escribir SQL).
- **Construcción y dependencias:** Maven (usando el wrapper `mvnw`).
- **Organización:** arquitectura por **capas** — Controller → Service → Repository, con **DTO** y **Mapper** para transportar los datos.

Base de datos

- Usamos **MySQL / MariaDB** (con **XAMPP**), base de datos `tienda_pc`. **Hibernate** crea las tablas automáticamente a partir de las entidades.

Frontend

- **Librería de interfaz:** React 18. **Servidor de desarrollo / empaquetado:** Vite 5.
- **Navegación:** React Router (rutas públicas vs rutas protegidas del panel).
- **Gráficos del dashboard:** Recharts.
- **Peticiones HTTP:** fetch nativo, centralizado en `client.js` (agrega el token y hace el refresco automático).
- **Estado de sesión:** React Context (`AuthContext`).
- **Íconos:** Bootstrap Icons. **Avisos:** sistema de **toasts** propio.

Validaciones

- **Formato de entrada:** Bean Validation (Jakarta) con `@Valid` y anotaciones (`@NotBlank`, `@Size`, `@Email`, `@Pattern`) en los DTOs de request.
- **Reglas de negocio / datos únicos:** métodos `existsBy...` de los repositorios; si algo está duplicado, el Service lanza un error **409**.
- **Errores uniformes:** un `@RestControllerAdvice` (`GlobalExceptionHandler`) convierte cualquier error en un JSON `{ "error": "..." }`, y el front lo muestra como **toast**.

Integraciones • Reportes • Imágenes • Pruebas

- **Datos reales de personas (RENIEC):** consumimos la API **apiperu.dev** por DNI, usando **RestClient** de Spring (con **timeouts**); es *best-effort* (si falla, no rompe el sistema).
- **Reporte PDF:** usamos la librería **OpenPDF** para armar el PDF de pedidos (tabla + totales).
- **Imágenes:** se guardan en el **sistema de archivos** del proyecto (`uploads/productos/`), se sirven en `/images/**` con un **resource handler**, y la subida usa **multipart** de Spring. En la BD solo va la ruta.
- **Pruebas:** JUnit 5 + Spring Boot Test (MockMvc) sobre una BD **H2** en memoria; **spring-security-test** para simular un usuario autenticado.
- **Datos de prueba:** un **CommandLineRunner** (`DataSeeder`) que carga la BD al arrancar, de forma idempotente.

1

Arquitectura y modelo de datos

Responsable: _____

El cimiento sobre el que trabajan todos: cómo se organiza el código en capas y cómo las clases de Java se convierten en las tablas de la base de datos.

🔗 ¿Qué hace?

Define la **estructura del proyecto** (capas del backend) y el **modelo de datos** (las entidades). Al arrancar el sistema, Hibernate lee las entidades y **crea las tablas** automáticamente. Todos los demás módulos se apoyan en esta base.

📄 Flujo — cómo nacen las tablas y cómo viaja una petición

A) Al arrancar el backend

- 1 Spring Boot inicia (**GamerStoreApplication.main()**) y se conecta a la BD `tienda_pc` (config en `application.properties`).
- 2 **Hibernate** lee las clases `@Entity` de `model/` y **crea/actualiza las tablas** (producto, categoria, cliente, pedido, pedido_item, usuario, refresh_token).

B) El viaje de una petición (ejemplo genérico)

- 1 La **pantalla** (React) llama a una función de `endpoints.js`.
- 2 `client.js` hace el fetch a `/api/...` con el token.
- 3 El **Controller** recibe → el **Service** aplica reglas → el **Repository** toca la BD.
- 4 El **Mapper** convierte la Entidad en un **DTO** → el Controller devuelve **JSON** → la pantalla se actualiza.

★ **¿Por qué DTO y no la entidad directa?** El **DTO** lleva solo los campos que la pantalla necesita. Ejemplo: al listar productos NO enviamos el objeto `Categoria` completo, solo `categoriaNombre`. Así la API no depende de cómo esté hecha la tabla y no exponemos datos internos.

📁 Modelo de datos (entidades = tablas)

- ▶ **Producto** (producto): nombre (*único*), descripción, precio, imagen (*ruta*), stock → **pertenece a una** Categoría.
- ▶ **Categoría** (categoria): nombre (*único*) → **tiene muchos** Productos.
- ▶ **Cliente** (cliente): dni (*único*), nombres, apellidos, email, teléfono, dirección.
- ▶ **Pedido** (pedido): fecha, estado, total, métodoPago → **pertenece a un** Cliente y **tiene muchos** PedidoItem.
- ▶ **PedidoItem** (pedido_item): cantidad, precioUnitario → une un Pedido con un Producto (una línea de la venta).
- ▶ **Usuario** (usuario): username (*único*), email (*único*), password (*BCrypt*), rol.
- ▶ **RefreshToken** (refresh_token): token, expiraEn, revocado → para el manejo de sesión.

📁 Carpetas y archivos

- ▶ `src/main/java/com/gamerstore/app/model/` — las entidades.
- ▶ `src/main/resources/application.properties` — conexión a la BD y configuración.
- ▶ `.../controller · service · repository · dto · mapper` — las capas del backend.
- ▶ `GamerStoreApplication.java` — arranque del sistema.

👁️ Qué mostrar en el código

model/Producto.java: `@Entity, @Column(unique=true)` en nombre, y `@ManyToOne` a Categoría (la relación). · La estructura de carpetas (las 6 capas) para mostrar la separación por responsabilidades.

▶ **Demo:** abre **phpMyAdmin** (XAMPP) → BD **tienda_pc** → muestra las tablas y cómo se relacionan (por ejemplo, `pedido_item` apunta a `pedido` y a `producto`).

? Preguntas típicas + respuesta

¿Quién crea las tablas?

Hibernate, a partir de las anotaciones @Entity/@Column de las entidades. No escribimos el CREATE TABLE a mano.

¿Qué es una relación @ManyToOne / @OneToMany?

Muchos productos pertenecen a una categoría (@ManyToOne); una categoría tiene muchos productos (@OneToMany). En la tabla se guarda la llave foránea categoria_id.

¿Para qué sirven las capas (Controller/Service/Repository)?

Para separar responsabilidades: el Controller atiende la web, el Service tiene las reglas, el Repository habla con la BD. Así es más ordenado, se prueba mejor y se cambia sin romper todo.

2

Datos reales + imágenes en el proyecto

Responsable: _____

De dónde salen los datos que se ven en el panel (la carga inicial) y cómo se guardan las imágenes de los productos dentro del proyecto, no en la base de datos.

🔗 ¿Qué hace?

Al arrancar, el sistema **llena la base de datos** con datos reales de tecnología (categorías, productos con precios en soles, clientes y pedidos históricos). Además, las **imágenes de los productos** se almacenan como archivos en el proyecto y en la BD solo se guarda su **ruta**.

🔄 Flujo A — Carga inicial de datos (al arrancar)

- 1 Arranca **DataSeeder** (un CommandLineRunner, se ejecuta 1 vez). Es **idempotente**: solo siembra si la tabla está vacía (`count() == 0`).
- 2 Inserta **10 categorías** y **28 productos** (con la ruta de su imagen local y precio en S/).
- 3 Para cada cliente, consulta su **DNI en RENIEC** (apiperu.dev) y usa el **nombre real**; si la API falla, usa un nombre de respaldo.
- 4 Genera **~40 pedidos** con fechas repartidas en los últimos 6 meses (para que el dashboard tenga historia).

★ **Ejemplo real:** al sembrar el cliente con DNI **70123456**, el sistema llama a RENIEC y guarda **"FIORELLA DE LA SOTA CASTRO"** (nombre real), no un nombre inventado.

🔄 Flujo B — Subir la imagen de un producto

- 1 **AdminProductos.jsx** → al elegir un archivo, `subirImagen()` → **AdminAPI.subirImagen(formData)** → `POST /api/admin/uploads` (multipart, con token).
- 2 **UploadController.subir()** valida que sea imagen y ≤ 5 MB, la guarda con un nombre único (UUID) en `uploads/productos/`.
- 3 Devuelve la ruta: **UploadResponse** { `url: "/images/productos/a1b2c3.jpg"` }.
- 4 El front guarda esa **ruta** en el formulario y muestra la **vista previa**. Al guardar el producto, en la BD solo se guarda ese texto.

★ **¿Y cómo se ve la imagen después?** El navegador pide `/images/productos/a1b2c3.jpg`; un **resource handler** en **WebConfig** la sirve desde la carpeta del proyecto. Nunca pasa por la base de datos.

📁 Carpetas y archivos

- ▶ `.../config/DataSeeder.java` — la carga inicial de datos.
- ▶ `.../controller/UploadController.java` — recibe y guarda la imagen.
- ▶ `.../config/WebConfig.java` — sirve las imágenes en `/images/**`.
- ▶ `uploads/productos/` — las imágenes físicas (dentro del proyecto).

👤 Qué mostrar en el código

DataSeeder.java: el `if (count() == 0)` y la lista de productos con `/images/productos/...` · **UploadController.java:** cómo genera el nombre con UUID y usa `Files.copy(...)`.

▶ **Demo:** en Productos → Nuevo → sube una imagen (aparece la vista previa) → guarda; luego abre la carpeta `uploads/productos/` y muestra el archivo nuevo.

? Preguntas típicas + respuesta

¿Por qué la imagen no va en la base de datos?

Guardar archivos binarios en la BD la hace pesada y lenta. Guardamos el **archivo en el proyecto** y en la BD solo la **ruta** (un texto corto).

¿El seeder duplica los datos si reinicio?

No. Solo siembra cuando la tabla está vacía (`count() == 0`), así que es seguro reiniciar.

¿Cómo validan la imagen subida?

El **UploadController** revisa que el tipo empiece por `image/` y que no supere 5 MB; además el nombre lo genera el servidor (UUID), no el usuario.

3

Seguridad: login JWT + sesión

Responsable: _____

Cómo se protege el panel: el login entrega un token, cada petición lo lleva, y la sesión se renueva sola sin sacar al usuario.

🔗 ¿Qué hace?

Da **autenticación real**. Sin iniciar sesión no se puede entrar al panel ni llamar a los endpoints `/api/admin/**`. El token de acceso dura poco (5 min) y se **renueva automáticamente** con un *refresh token*; un contador muestra el tiempo restante.

🔄 Flujo A — Iniciar sesión

- 1 **Login.jsx** → `submit()` → **AuthContext.login()** → **AuthAPI.login()** → `POST /api/auth/login` con `{username, password}`.
- 2 **AuthController.login()** valida con **AuthenticationManager** + **CustomUserDetailsService** (busca en tabla `usuario`) y compara la clave con **BCrypt**.
- 3 Si es correcto: **JwtService.generarAccess()** firma el **access token** (5 min) y **RefreshTokenService.crear()** guarda un **refresh token** en la tabla `refresh_token`.
- 4 Devuelve **LoginResponse** `{accessToken, refreshToken, username, nombre, rol}`; el front lo guarda en `localStorage`.

★ **Ejemplo:** ingresas `admin123 / gamerstore123` → recibes un access token (un texto largo tipo `eyJhbGciOi...`) que dice "soy admin123, rol ADMIN, vence en 5 min".

🔄 Flujo B — Cada petición al panel

- 1 `client.js` agrega la cabecera `Authorization: Bearer <token>` a la petición.
- 2 **JwtAuthenticationFilter** (en el backend) valida la firma y la expiración del token y marca al usuario como autenticado.
- 3 **SecurityConfig** comprueba que `/api/admin/**` exige rol **ADMIN**. Si el token es válido → pasa; si no → **401**.

🔄 Flujo C — Refresco silencioso (lo más ingenioso)

- 1 Pasados 5 min, una petición devuelve **401** (el access venció).
- 2 `client.js` lo detecta y llama **una sola vez** a `POST /api/auth/refresh` con el refresh token.
- 3 **RefreshTokenService.rotar()** valida el refresh en la BD, lo **revoca** y crea uno nuevo (rotación). Devuelve tokens nuevos.
- 4 `client.js` guarda los nuevos y **reintenta la petición original** — el usuario nunca se entera. Si el refresh ya no vale → recién ahí va a `/admin/login`.

★ **Ejemplo:** estás editando un producto a los 6 minutos. Guardas → el access venció → el sistema pide uno nuevo por detrás y tu guardado se completa igual, sin pedirte volver a logear.

🔄 Flujo D — Recargar (F5) / Cerrar sesión / Contador

- 1 **Restaurar sesión:** al recargar, **AuthContext** llama `GET /api/auth/me` → **AuthController.me()** devuelve el usuario → la sesión sigue.
- 2 **Cerrar sesión:** **Sidebar.jsx** → `POST /api/auth/logout` → **RefreshTokenService.revocar()** anula el refresh en la BD; el front limpia `localStorage`.
- 3 **Contador:** **SessionTimer.jsx** lee el campo `exp` dentro del token y muestra la cuenta atrás (no hace peticiones).

📁 Carpetas y archivos

- ▶ `.../config/security/` — **JwtService**, **JwtAuthenticationFilter**, **SecurityConfig**, **CustomUserDetailsService**, **RefreshTokenService**.
- ▶ `.../controller/AuthController.java` — login, refresh, logout, me.
- ▶ `frontend/src/api/client.js` — token + refresco automático · `auth/AuthContext.jsx` · `components/admin/SessionTimer.jsx` · `pages/admin/Login.jsx`.

Qué mostrar en el código

SecurityConfig.java: la lista `authorizeHttpRequests(...)` con `permitAll()` vs `hasRole("ADMIN")`. · **client.js**: el bloque que, ante un 401, llama a *refresh* y reintenta.

► **Demo**: DevTools → Application → Local Storage: muestra `gs_token` y `gs_refresh`. En el panel, al minuto final el contador se pone ámbar y, al pasar los 5 min, la sesión se renueva sola.

? Preguntas típicas + respuesta

¿Qué es un JWT?

Un token firmado que lleva el usuario y su rol. El servidor verifica la firma en cada petición y no necesita guardar sesión en memoria (stateless).

¿Por qué access corto (5 min) + refresh largo?

Si roban el access, expira rápido. El refresh vive en la BD y se puede **revocar**, lo que permite cerrar sesión de verdad.

¿Dónde se protege el panel?

En **SecurityConfig**: `/api/admin/**` exige el rol ADMIN. Sin token válido, la API responde 401 y el front manda al login.

¿Cómo guardan las contraseñas?

Con **BCrypt** (hash de una vía). Nunca en texto plano; ni nosotros podemos “ver” la contraseña original.

4

Productos y Categorías

Responsable: _____

El corazón del catálogo: crear, editar y eliminar productos y categorías, con validación de nombres únicos y las reglas que protegen los datos.

🔗 ¿Qué hace?

Permite administrar el **catálogo**: el CRUD (crear, listar, editar, eliminar) de **productos** y **categorías**. Evita nombres duplicados y no deja borrar una categoría que tiene productos.

🔄 Flujo — Crear un producto

- 1 **AdminProductos.jsx** → `guardar()` arma `{nombre, descripcion, precio, stock, imagen, categoriaId}` → **AdminAPI.crearProducto()** → `POST /api/admin/productos`.
- 2 **AdminProductoController.crear()** valida el formato con `@Valid` → **ProductoService.crear()**.
- 3 **Regla: `ProductoRepository.existsByNombreIgnoreCase()`** → si ya existe, lanza **409** "Ya existe un producto con ese nombre".
- 4 Si no existe → asocia la categoría y **ProductoRepository.save()** → tabla `producto`.
- 5 Vuelve como **ProductoDTO** (JSON) → **UI**: toast "Producto creado" y la tabla se refresca.

🌟 **Ejemplo:** intentas crear "NVIDIA GeForce RTX 4070" pero ya existe → el backend responde 409 y el front muestra un **toast rojo** "Ya existe un producto con ese nombre"; el producto NO se crea.

🔄 Flujo — Listar / Editar / Eliminar / Categorías

- 1 **Listar:** `GET /api/admin/productos` → **ProductoService.todos()** → **ProductoRepository.findAll()** → tabla.
- 2 **Editar:** `PUT .../{id}` → **actualizar()** [valida con `existsByNombreIgnoreCaseAndIdNot`].
- 3 **Eliminar:** pide confirmación → `DELETE .../{id}` → `deleteById()`.
- 4 **Categorías:** mismo patrón (`/api/admin/categorias`). Al **eliminar**, si `existsByCategoriaId()` es true (la categoría tiene productos) → **bloquea** el borrado.

🌟 **Nota:** **buscar, ordenar y paginar** la tabla NO llama al backend: lo hace el hook **useTableControls.js** en el navegador, sobre la lista ya cargada (es instantáneo).

📁 Carpetas y archivos

- ▶ `frontend/src/pages/admin/AdminProductos.jsx` · `AdminCategorias.jsx`
- ▶ `.../controller/AdminProductoController.java` · `AdminCategoriaController.java`
- ▶ `.../service/ProductoService.java` · `CategoriaService.java` · `.../repository/...`

🔗 Qué mostrar en el código

ProductoService.crear(): la línea `existsByNombreIgnoreCase` que lanza el 409. · **CategoriaService.eliminar():** el bloqueo con `existsByCategoriaId`.

▶ **Demo:** crea un producto con un nombre que ya existe (sale el toast rojo), crea uno nuevo válido, editálo, y muestra que no puedes borrar una categoría que tiene productos.

? Preguntas típicas + respuesta

¿Dónde evitan los nombres duplicados?

En el **Service**, con `existsByNombreIgnoreCase` del repositorio; si existe, lanzamos un 409 que el front muestra como toast.

¿La búsqueda de la tabla pega al servidor cada vez?

No. Traemos la lista una vez y filtramos/ordenamos en el navegador con **useTableControls**. Es más rápido y no recarga la API.

¿Por qué no se puede borrar una categoría con productos?

Porque los productos quedarían "huérfanos". El Service lo verifica con `existsByCategoriaId` y bloquea el borrado.

La parte de ventas: clientes con datos reales por DNI, el registro de pedidos con cálculo del total, y la descarga del reporte en PDF.

🔗 ¿Qué hace?

Gestiona **clientes** (autocompletando nombres reales desde RENIEC por DNI), registra **ventas/pedidos** calculando el total, y genera un **reporte PDF** de pedidos con filtros.

🔄 Flujo — Buscar cliente por DNI (RENIEC)

- 1 **AdminClientes.jsx** → `buscarDni()` (valida 8 dígitos) → **AdminAPI.buscarDni(dni)** → `GET /api/admin/clientes/reniec/{dni}`.
- 2 **AdminClienteController.reniec()** → **ReniecService.consultarDni()** llama a **apiperu.dev** con **RestClient** (con timeouts) usando el token.
- 3 *Best-effort*: si la API responde → **ReniecPersona** {nombres, apellidos}; si falla → **404** sin romper nada.
- 4 **UI**: autocompleta nombres y apellidos + toast "Datos obtenidos de RENIEC". Al guardar, valida DNI y email únicos.

★ **Ejemplo**: escribes **70123456** y pulsas "Buscar" → se llenan automáticamente "FIORELLA" / "DE LA SOTA CASTRO"; solo completas teléfono y dirección.

🔄 Flujo — Registrar un pedido

- 1 En **AdminPedidos.jsx** eliges cliente y agregas productos con `agregarItem()` (el total se calcula en el front mientras armas la venta).
- 2 Al enviar → **guardar()** manda {clienteId, metodoPago, items:[{productoId, cantidad}]} → `POST /api/admin/pedidos`.
- 3 **PedidoService.crear()**: busca el cliente; por cada ítem busca el producto, crea un **PedidoItem** con el precio actual y **suma el subtotal**.
- 4 **PedidoRepository.save()** guarda el `pedido` y sus `pedido_item` juntos (cascade). El total queda calculado en el backend.

★ **Ejemplo con números**: vendes 2× "RTX 4070" (S/2,599) + 1× "Ryzen 5 5600" (S/549). El front manda `items:[{2,2},{5,1}]`. El Service calcula $2 \times 2599 + 1 \times 549 = \text{S/5,747}$ y guarda el pedido con ese total.

🔄 Flujo — Descargar el reporte PDF

- 1 **descargarPDF()** arma la URL con los filtros (fecha/estado) → **downloadBlob()** hace `GET /api/admin/pedidos/reporte.pdf?...` con el token.
- 2 **AdminPedidoController.reporte()** → **PedidoService.reporte()** filtra los pedidos → **PedidoReporteService.generar()** arma el PDF con **OpenPDF** (encabezado, tabla, total).
- 3 El backend devuelve el archivo (Content-Disposition: attachment) → el navegador lo **descarga** como `reporte-pedidos.pdf`.

★ **Extra**: el estado del pedido (PENDIENTE, PAGADO, ENVIADO, ENTREGADO, CANCELADO) se cambia con el lápiz → `PUT .../{id}` y el badge de la tabla cambia de color.

📁 Carpetas y archivos

- ▶ `frontend/src/pages/admin/AdminClientes.jsx` · `AdminPedidos.jsx`
- ▶ `.../controller/AdminClienteController.java (/reniec/{dni})` · `AdminPedidoController.java (/reporte.pdf)`
- ▶ `.../service/ClienteService` · `PedidoService` · `ReniecService` · `PedidoReporteService`

👁️ Qué mostrar en el código

ReniecService.consultarDni(): el llamado con RestClient, los **timeouts** y el try/catch best-effort. · **PedidoService.crear()**: el bucle que suma el subtotal. · **PedidoReporteService.generar()**: cómo arma la tabla del PDF.

► **Demo:** Clientes → Nuevo → DNI **70123456** → “Buscar” (se autocompleta). Pedidos → arma una venta con 2–3 productos (mira el total) → “Registrar” → “Descargar PDF”.

? Preguntas típicas + respuesta

¿De dónde salen los nombres reales?

De la API **apiperu.dev** (RENIEC), consultando por DNI desde **ReniecService**.

¿Qué pasa si esa API se cae?

Nada grave: es *best-effort* con timeouts. Si falla, el flujo continúa (el seeder usa un nombre de respaldo; el formulario muestra un aviso).

¿Dónde se calcula el total del pedido?

En el **PedidoService**, en el backend: por cada ítem hace $\text{precio} \times \text{cantidad}$ y suma. No se confía en el total que manda el front.

¿Cómo generan el PDF?

Con la librería **OpenPDF** en **PedidoReporteService**; el endpoint devuelve el archivo para descarga.

Lo que da "terminado" al sistema: el tablero con indicadores, la gestión de usuarios del panel y cómo los errores/duplicados llegan al usuario como avisos claros.

🔗 ¿Qué hace?

Muestra el **dashboard** (indicadores del negocio y productos más vendidos), administra los **usuarios** del sistema (con cifrado de contraseña y protección del último admin), y centraliza el **manejo de errores** que se muestran como toasts.

🔄 Flujo — Cargar el Dashboard

- 1 **Dashboard.jsx** → **AdminAPI.dashboard()** → `GET /api/admin/dashboard`.
- 2 **AdminDashboardController** junta: `count()` de cada repositorio; **PedidoRepository.sumTotal()** (ventas); **findByStockLessThanEqual...** (stock bajo); **PedidoRepository.topProductos()** (agrupa `pedido_item` por producto y suma cantidades).
- 3 Devuelve **DashboardDTO** {totales, totalVentas, stockBajo[], topProductos[]}
- 4 **UI**: tarjetas KPI, gráficos (Recharts) y el panel "Más vendidos".

🔄 Flujo — Usuarios del sistema

- 1 **Listar/Crear/Editar/Eliminar** contra `/api/admin/usuarios` → **UsuarioService**.
- 2 **Reglas**: `existsByUsername()` / `existsByEmail()` → si están repetidos, error; la contraseña se **hashea con BCrypt** antes de guardar.
- 3 **Protección**: al eliminar, si es el **último ADMIN** (`countByRol(ADMIN) <= 1`) → lo bloquea, para no quedarnos sin acceso.

🔴 **Ejemplo**: intentas eliminar al único administrador → el sistema responde "No puedes eliminar el último administrador" y no lo borra.

🔄 Flujo — Cómo un error/duplicado llega al usuario (transversal)

- 1 El **Service** lanza una excepción con mensaje (ej. 409 "La categoría ya existe").
- 2 **GlobalExceptionHandler** (@RestControllerAdvice) la convierte en { "error": "La categoría ya existe" } con el código correcto.
- 3 `client.js` ve que no es 2xx, lee el error y lanza `Error(mensaje)`.
- 4 La pantalla, en el catch, hace **toast.error(mensaje)** → aparece el aviso rojo. El registro NO se crea.

🔴 **Códigos que usa la API**: 400 datos inválidos · 401 no autenticado · 404 no encontrado · 409 duplicado o en uso · 413 imagen muy grande.

📁 Carpetas y archivos

- ▶ `frontend/src/pages/admin/Dashboard.jsx` · `AdminUsuarios.jsx` · `components/ui/Toast.jsx`
- ▶ `.../controller/AdminDashboardController` · `AdminUsuarioController` · `GlobalExceptionHandler`
- ▶ `.../service/UsuarioService.java` (+ las validaciones `existsBy...` en los demás services)

👁️ Qué mostrar en el código

GlobalExceptionHandler.java: cada `@ExceptionHandler` y a qué status mapea. · **UsuarioService.eliminar()**: el if que protege al último admin. · **Dashboard.jsx**: cómo usa `data.topProductos`.

▶ **Demo**: intenta crear una categoría repetida (sale el toast rojo), muestra el dashboard con los gráficos y "Más vendidos", y crea un usuario nuevo desde el módulo Usuarios.

? Preguntas típicas + respuesta

¿Cómo evitan datos duplicados en todo el sistema?

Con métodos existsBy... en los repositorios; si existe, el Service lanza un 409 y el **GlobalExceptionHandler** lo convierte en un mensaje que se ve como toast.

¿De dónde salen los "más vendidos"?

De una consulta (`PedidoRepository.topProductos`) que agrupa las líneas de pedido por producto y suma las cantidades.

¿Por qué protegen al último admin?

Para no quedarnos sin acceso al panel. Si solo queda un ADMIN, el sistema no permite eliminarlo.



Apéndice — Cómo levantar el proyecto, estructura y glosario

Para la demo y para entender los términos

► Cómo levantar el proyecto (para el que hace la demo)

- 1 Iniciar **MySQL** en **XAMPP** (la BD `tienda_pc` se crea sola si no existe).
- 2 **Backend**: en la raíz del proyecto, ejecutar `mvnw.cmd spring-boot:run` → levanta en `:8080` y carga los datos de prueba.
- 3 **Frontend**: en la carpeta `frontend/`, ejecutar `npm install` (la primera vez) y luego `npm run dev` → abre en `:5173`.
- 4 Entrar al panel en `http://localhost:5173/admin` con **admin123 / gamerstore123**.

Estructura de carpetas

```
gamerstore-main/
├─ src/main/java/com/gamerstore/app/
│  └─ model/          (entidades = tablas)
│  └─ repository/     (acceso a datos, Spring Data JPA)
│  └─ service/        (reglas de negocio, validaciones)
│  └─ controller/     (endpoints /api/...)
│  └─ dto/ mapper/    (objetos de transferencia + conversión)
│  └─ config/         (seeder, seguridad JWT, web)
├─ src/main/resources/application.properties
├─ src/test/...       (pruebas de integración)
├─ uploads/productos/ (imágenes guardadas en el proyecto)
└─ frontend/src/
   └─ pages/admin/    (pantallas del panel)
   └─ components/     (navbar, tabla, toast, modal...)
   └─ api/            (client.js = fetch + token; endpoints.js)
   └─ auth/           (contexto de sesión + ruta protegida)
   └─ utils/ hooks/   config/
```

Glosario (por si preguntan un término)

- **API REST**: conjunto de URLs (`/api/...`) por donde el front pide y envía datos en formato JSON.
- **Endpoint**: una de esas URLs con su método (GET/POST/PUT/DELETE), p. ej. `POST /api/admin/productos`.
- **JWT (JSON Web Token)**: un token firmado que identifica al usuario; el servidor lo verifica sin guardar sesión (stateless).
- **Stateless**: el servidor no guarda la sesión en memoria; toda la info va en el token de cada petición.
- **Hash / BCrypt**: transformar la contraseña en un texto irreversible; se compara, pero no se puede “des-hacer”.
- **ORM (JPA/Hibernate)**: traduce entre objetos de Java (entidades) y tablas de la base de datos, sin escribir SQL a mano.
- **DTO**: objeto simple que lleva solo los datos que la pantalla necesita (lo que viaja como JSON).
- **Mapper**: el que convierte una Entidad en su DTO.
- **CRUD**: Crear, Leer (listar), Actualizar y Eliminar — las 4 operaciones básicas de un módulo.
- **Cascade**: al guardar/borrar un Pedido, sus líneas (PedidoItem) se guardan/borran con él automáticamente.
- **Multipart**: el formato con el que el navegador sube archivos (las imágenes).
- **Best-effort**: “se intenta, pero si falla no rompe nada” (así consultamos RENIEC).

 **Cierre**: si dominan el **viaje de una petición** (portada) y esta chuleta de “**qué usamos para qué**”, pueden explicar y defender cualquier parte del ERP.